

```

"""
optimise.py - Bayesian optimisation of cabin seat RMS using trained surrogate.

Uses BoTorch (preferred) or falls back to bayes_opt.

Usage
-----
python src/optimise.py \
    --ckpt checkpoints/best.pt \
    --norms checkpoints/norm_stats.npz \
    --out outputs/opt/

Optional (verify optimal with ODE):
python src/optimise.py --ckpt checkpoints/best.pt --norms checkpoints/norm_stats.
                        --verify_ode
"""

from __future__ import annotations

import argparse
import json
import sys
import warnings
from pathlib import Path
from typing import Dict

import numpy as np
import pandas as pd
import torch

sys.path.insert(0, str(Path(__file__).parent))

from config import BASE_CFG, BAYES_CFG, DT, PARAM_BOUNDS, PARAM_NAMES, TRAIN_CFG
from dataset import NormStats
from model import PhysicsLSTM
from physics import (
    build_road_signals, compute_seat_rms,
    precompute_road_array, run_one_case, static_equilibrium,
)

try:
    from botorch.models import SingleTaskGP
    from botorch.fit import fit_gpytorch_mll
    from botorch.acquisition import LogExpectedImprovement

```

```

    from botorch.optim import optimize_acqf
    from gpytorch.mlls import ExactMarginalLogLikelihood
    HAS_BOTORCH = True
except ImportError:
    HAS_BOTORCH = False

try:
    from bayes_opt import BayesianOptimization
    HAS_BAYESOPT = True
except ImportError:
    HAS_BAYESOPT = False

# -----
# RMS emulator (wraps model, caches road tensor)
# -----

class RMSEmulator:
    """
    Evaluates cabin seat RMS for a given parameter vector using
    a forward pass of the trained surrogate (~ms per call).

    The road tensor is cached - only the 8 design parameters change
    between calls during optimisation.
    """

    def __init__(self,
                 model: PhysicsLSTM,
                 road_norm: np.ndarray, # [T, 6] normalised
                 ns: NormStats,
                 cfg: Dict,
                 device: str = "cpu"):
        self.model = model.eval().to(device)
        self.device = device
        self.cfg = cfg
        self.h_seat = float(cfg.get("hcp", 0.1)) # ISO 2631: use hcp as lever a
        self.ns = ns
        # Cache road tensor
        self.road_t = torch.tensor(road_norm, dtype=torch.float32,
                                   device=device).unsqueeze(0) # [1, T, 6]

        # Pre-compute transient skip index
        self.n_skip = max(0, int(0.5 / (DT * TRAIN_CFG.get("downsample", 4))))

    @torch.no_grad()
    def __call__(self, params: Dict) -> float:
        """

```

```

params: dict with physical-unit values for PARAM_NAMES keys.
Returns: cabin seat RMS [m/s2]
"""
p_raw = np.array([float(params[k]) for k in PARAM_NAMES], np.float32)
p_norm = (p_raw - self.ns.p_lo) / (self.ns.p_hi - self.ns.p_lo + 1e-8)
p_t = torch.tensor(p_norm, dtype=torch.float32,
                    device=self.device).unsqueeze(0) # [1, 8]

_, ap = self.model(self.road_t, p_t) # [1, T, 3]
a = ap[0].cpu().numpy() # [T, 3] normalised
# Denormalise
a_phys = a * self.ns.a_std + self.ns.a_mean # [T, 3] physical units
a_phys = a_phys[self.n_skip:]

# ISO 2631 3-axis combined RMS
h = self.h_seat # reuse h_seat as hcp lever arm
az = a_phys[:, 0]
ax = -h * a_phys[:, 1] # pitch → longitudinal
ay = h * a_phys[:, 2] # roll → lateral
return float(np.sqrt(np.mean(az**2) + np.mean(ax**2) + np.mean(ay**2)))

def normalise_params(self, p_dict: Dict) -> np.ndarray:
    p = np.array([float(p_dict[k]) for k in PARAM_NAMES], np.float32)
    return (p - self.ns.p_lo) / (self.ns.p_hi - self.ns.p_lo + 1e-8)

def denormalise_params(self, p_norm: np.ndarray) -> Dict:
    p_phys = self.ns.p_lo + p_norm * (self.ns.p_hi - self.ns.p_lo)
    return {k: float(p_phys[j]) for j, k in enumerate(PARAM_NAMES)}

# -----
# BoTorch optimiser (primary)
# -----

def run_botorch(emulator: RMSEmulator,
                n_init: int = BAYES_CFG["n_init"],
                n_iter: int = BAYES_CFG["n_iter"],
                seed: int = BAYES_CFG["seed"]) -> Dict:
    from scipy.stats import qmc
    torch.manual_seed(seed)
    dim = len(PARAM_NAMES)
    bounds = torch.zeros(2, dim, dtype=torch.double)
    bounds[1, :] = 1.0 # all params normalised to [0, 1]

    def eval_norm(x_np: np.ndarray) -> float:
        return emulator(emulator.denormalise_params(x_np.astype(np.float32)))

```

```

# Sobol initial design
sobol = qmc.Sobol(d=dim, scramble=True, seed=seed)
X0 = sobol.random(n_init).astype(np.float64)
Y0 = np.array([[eval_norm(x)] for x in X0])
print(f" Initial {n_init} Sobol samples: "
      f"RMS in [{Y0.min():.4f}, {Y0.max():.4f}]" )

train_X = torch.tensor(X0, dtype=torch.double)
train_Y = -torch.tensor(Y0, dtype=torch.double) # maximise -RMS

best_rms = float(Y0.min())
best_x    = X0[int(Y0.argmin())]

for it in range(n_iter):
    try:
        gp = SingleTaskGP(train_X, train_Y)
        mll = ExactMarginalLogLikelihood(gp.likelihood, gp)
        fit_gpytorch_mll(mll)
        acqf = LogExpectedImprovement(gp, best_f=train_Y.max())
        cand, _ = optimize_acqf(acqf, bounds=bounds, q=1,
                                num_restarts=10, raw_samples=256)
        x_new = cand[0].detach().numpy().astype(np.float64)
    except Exception as e:
        print(f" BoTorch iter {it+1} failed ({e}); random fallback")
        x_new = np.random.default_rng(seed + it).random(dim)

    y_new = eval_norm(x_new)
    print(f" iter {it+1:3d}/{n_iter} RMS={y_new:.5f} "
          f"best={min(best_rms, y_new):.5f}")

    train_X = torch.cat([train_X,
                          torch.tensor(x_new, dtype=torch.double).unsqueeze(0)
    ])
    train_Y = torch.cat([train_Y,
                          torch.tensor([-y_new], dtype=torch.double)])

    if y_new < best_rms:
        best_rms = y_new
        best_x    = x_new

best_params = emulator.denormalise_params(best_x.astype(np.float32))
return {"params": best_params, "rms_surrogate": best_rms,
        "history_X": train_X.numpy(), "history_Y": -train_Y.numpy()}

# -----
# bayes_opt fallback
# -----

```

```

def run_bayesopt_fallback(emulator: RMSEmulator,
                          n_init: int = BAYES_CFG["n_init"],
                          n_iter: int = BAYES_CFG["n_iter"],
                          seed: int = BAYES_CFG["seed"]) -> Dict:
    if not HAS_BAYESOPT:
        raise ImportError("pip install bayesian-optimization")

    def objective(**kwargs):
        p_norm = np.array([kwargs[k] for k in PARAM_NAMES], np.float32)
        p_phys = emulator.denormalise_params(p_norm)
        return -emulator(p_phys)

    pbounds = {k: (0.0, 1.0) for k in PARAM_NAMES}
    opt = BayesianOptimization(f=objective, pbounds=pbounds,
                              random_state=seed, verbose=2)
    opt.maximize(init_points=n_init, n_iter=n_iter)

    best_norm = np.array([opt.max["params"][k] for k in PARAM_NAMES], np.float32)
    best_params = emulator.denormalise_params(best_norm)
    return {"params": best_params, "rms_surrogate": -opt.max["target"]}

# -----
# Optional ODE verification of optimal point
# -----

def verify_with_ode(best_params: Dict, cfg_base: Dict,
                    downsample: int = 4) -> float:
    print("\n=== ODE verification of optimal parameters ===")
    t_eval = np.arange(0.0, float(cfg_base["sim_duration_s"]) + DT, DT)[::downsam
    df = run_one_case(best_params, cfg_base, t_eval, verbose=True)
    if df is None:
        print(" ODE verification FAILED")
        return float("nan")
    rms = compute_seat_rms(df, {**cfg_base, **best_params})
    print(f" ODE verified RMS = {rms:.5f} m/s2")
    return rms

# -----
# Main optimisation entry point
# -----

def optimise(
    ckpt_path: str,
    norms_path: str,

```

```

    out_dir:      str,
    cfg_base:     Dict = None,
    train_cfg:    Dict = None,
    bayes_cfg:    Dict = None,
    verify_ode:   bool = False,
) -> Dict:

    cfg_base = cfg_base or BASE_CFG
    train_cfg = train_cfg or TRAIN_CFG
    bayes_cfg = bayes_cfg or BAYES_CFG
    out_path = Path(out_dir); out_path.mkdir(parents=True, exist_ok=True)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    ds = int(train_cfg.get("downsample", 4))

    # — load model —————
    ns = NormStats.load(norms_path)
    model = PhysicsLSTM(
        road_dim=train_cfg["road_dim"],
        param_dim=train_cfg["param_dim"],
        lstm_hidden=train_cfg["lstm_hidden"],
        lstm_layers=train_cfg["lstm_layers"],
        dropout=0.0,
    )
    ckpt = torch.load(ckpt_path, map_location=device)
    model.load_state_dict(ckpt["model"])
    print(f"Loaded surrogate from {ckpt_path}")

    # — pre-compute road —————
    t_eval = np.arange(0.0, float(cfg_base["sim_duration_s"]) + DT, DT)[::ds]
    road_arr = precompute_road_array(cfg_base, t_eval)
    road_norm = (road_arr - ns.r_mean) / ns.r_std

    emulator = RMSEmulator(model, road_norm, ns, cfg_base, device=device)

    # Baseline: evaluate at nominal parameters
    nominal_params = {k: float(cfg_base[k]) for k in PARAM_NAMES}
    rms_nominal = emulator(nominal_params)
    print(f"\nBaseline RMS (nominal params): {rms_nominal:.5f} m/s²")

    # — run Bayesian optimisation —————
    print(f"\nRunning Bayesian optimisation ")
    print(f"    (backend: {'BoTorch' if HAS_BOTORCH else 'bayes_opt'}) ...")
    print(f"    n_init={bayes_cfg['n_init']} n_iter={bayes_cfg['n_iter']} "
          f"dim={len(PARAM_NAMES)}\n")

    if HAS_BOTORCH:
        result = run_botorch(emulator,

```

```

        n_init=bayes_cfg["n_init"],
        n_iter=bayes_cfg["n_iter"],
        seed=bayes_cfg["seed"])
elif HAS_BAYESOPT:
    result = run_bayesopt_fallback(emulator,
                                   n_init=bayes_cfg["n_init"],
                                   n_iter=bayes_cfg["n_iter"],
                                   seed=bayes_cfg["seed"])
else:
    raise ImportError(
        "Install BoTorch: pip install botorch\n"
        "or: pip install bayesian-optimization")

best_params = result["params"]
rms_surr     = result["rms_surrogate"]
improvement = (rms_nominal - rms_surr) / rms_nominal * 100

# — print results —————
print("\n" + "="*55)
print("OPTIMAL PARAMETERS")
print("="*55)
print(f"{'Parameter':<15} {'Nominal':>12} {'Optimal':>12} {'Change':>8}")
print("-"*55)
for k in PARAM_NAMES:
    nom = float(cfg_base[k])
    opt = float(best_params[k])
    print(f"{k:<15} {nom:>12.4f} {opt:>12.4f} "
          f"{(opt-nom)/nom*100:>+7.1f}%")
print("-"*55)
print(f"{'Seat RMS':<15} {rms_nominal:>12.5f} {rms_surr:>12.5f} "
      f"{-improvement:>+7.1f}%")
print(f"\nRMS reduction: {improvement:.2f}% "
      f"({rms_nominal:.5f} → {rms_surr:.5f} m/s2)")

# — optional ODE verification —————
rms_ode = float("nan")
if verify_ode:
    rms_ode = verify_with_ode(best_params, cfg_base)

# — save results —————
out = {
    "nominal_params": nominal_params,
    "optimal_params": best_params,
    "rms_nominal": rms_nominal,
    "rms_surrogate": rms_surr,
    "rms_ode_verify": rms_ode,
    "rms_reduction_%": improvement,

```

```

    }
    with open(str(out_path / "optimal_params.json"), "w") as f:
        json.dump(out, f, indent=2)

    # Save as CSV for easy reading
    rows = []
    for k in PARAM_NAMES:
        rows.append({"parameter": k,
                    "nominal": float(cfg_base[k]),
                    "optimal": float(best_params[k]),
                    "change_%": (float(best_params[k]) - float(cfg_base[k]))
                               / float(cfg_base[k]) * 100})
    pd.DataFrame(rows).to_csv(str(out_path / "param_comparison.csv"), index=False)

    # Save BO history if available
    if "history_X" in result:
        hist_df = pd.DataFrame(result["history_X"],
                               columns=[f"p{i}" for i in range(len(PARAM_NAMES))])
        hist_df["rms"] = result["history_Y"].flatten()
        hist_df.to_csv(str(out_path / "bo_history.csv"), index=False)

    print(f"\nResults saved → {out_path}")
    return out

# -----
# CLI
# -----

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Bayesian optimisation of cabin seat RMS")
    parser.add_argument("--ckpt", required=True)
    parser.add_argument("--norms", required=True)
    parser.add_argument("--out", default="outputs/opt")
    parser.add_argument("--n_init", type=int, default=None)
    parser.add_argument("--n_iter", type=int, default=None)
    parser.add_argument("--verify_ode", action="store_true",
                        help="Verify optimal parameters with full ODE solve")
    parser.add_argument("--data_dir", type=str, default=None)
    args = parser.parse_args()

    cfg = dict(BASE_CFG)
    bc = dict(BAYES_CFG)
    if args.n_init: bc["n_init"] = args.n_init
    if args.n_iter: bc["n_iter"] = args.n_iter
    if args.data_dir:

```



```
for side in ("front_left", "front_right",
            "rear1_left", "rear1_right",
            "rear2_left", "rear2_right"):
    ax = "front" if "front" in side else ("rear1" if "1" in side else "re
    sd = "left" if "left" in side else "right"
    cfg[f"axle{ax}_{sd}_csv"] = f"{args.data_dir}/{side}.csv"

optimise(args.ckpt, args.norms, args.out,
        cfg_base=cfg, bayes_cfg=bc, verify_ode=args.verify_ode)
```